



Laboratory Report

LECTURER	John ORaw
STUDENT NAME	Liam Saunders
STUDENT NUMBER	L00187849
PROGRAMME	MSc Cloud
MODULE CODE	INFS_IT903
ASSIGNMENT TITLE	Academic Report - Ansible
SUBMISSION DATE	31/12/2025

Student Declaration

1. I have accurately identified and included the sources of all facts, ideas, opinions, and viewpoints from others in the assignment references. All direct quotations, paraphrasing, and discussions of ideas from books, journal articles, internet sources, course materials, or any other sources used are properly acknowledged and cited in the assignment references.
2. I have not used unauthorised artificial intelligence tools or aids.
3. I understand and am compliant with ATU's policy and procedures regarding Academic Integrity and I am aware of the consequences of any violations.
4. I have followed the referencing guidelines recommended in the assignment instructions and / or programme documentation.
5. By signing this form or by submitting material online I confirm that this assignment, or any part of it, has not been previously submitted by me or any other person for assessment on this or any other programme of study.
6. By signing this form or by submitting material for assessment online I confirm that I have read and understood [AQAE022 Academic Integrity Policy](#)

Student Signature	Date
Liam Saunders	31/12/2025

Contents

Table Of Figures	3
Description.....	4
Aims.....	4
Method	4
Results and Testing.....	6
Conclusions	6
References	7
Appendices.....	8

Table Of Figures

Figure 1: VMware NAT network configuration	8
Figure 2: Net plan static IP configuration	8
Figure 3: Disable Network Config file	9
Figure 4: /etc/hosts hostname resolution	9
Figure 5: Network connectivity test (ping)	10
Figure 6: SSH key generation	10
Figure 7: Ansible YAML inventory file.....	10
Figure 8: Update Playbook code	11
Figure 9: Install Apache Web Server Code.....	11
Figure 10: install MariaDB Code.....	12

Description

This laboratory report documents the configuration and use of Ansible as an automation tool for managing Linux servers in a local, virtualised environment. The work mainly focuses on the preparation of multiple Ubuntu 24.04 servers, the use of static networking, secure SSH-based authentication, and the deployment of Ansible for executing remote commands and playbooks. The laboratory demonstrates the foundational principles of Infrastructure as Code (IaC), including repeatability, idempotence, and configuration management at scale.

The environment was built using Azure Labs with a NAT-based virtual network, allowing multiple Linux instances to communicate securely without requiring external DNS infrastructure. Ansible was installed and configured on a dedicated management node, enabling the centralised administration of target servers. The laboratory culminates in the execution of Ansible playbooks to perform system updates and service installations, illustrating the practical application of automation concepts within modern DevOps workflows.

Aims

The aims of this laboratory were as follows:

1. To configure multiple Ubuntu Linux servers with static IP addresses and consistent hostnames
2. The laboratory also aimed to establish secure, password less SSH authentication between an Ansible control node and the managed servers.
3. The task also includes installing and configuring Ansible for remote command execution and playbook-driven automation.
4. The goal is to showcase Ansible's capability to consistently manage systems using playbooks.
5. The goal is to gain a complete understanding of Ansible's role in Infrastructure as Code and scalable system administration.

Method

The laboratory work was carried out within a local virtualised environment using Azure Labs Workstation, providing a controlled and repeatable platform for the configuration, deployment, and testing of Ansible automation. The environment consisted of multiple Ubuntu 24.04 Linux virtual machines connected through a NAT-based virtual network, allowing secure internal communication while maintaining limited external connectivity. This approach mirrors common industry practices where automation tools are tested within isolated environments. The laboratory aimed to force deployment to production systems (Red Hat 2024). Static IP configuration was used in Ansible manager for this lab as shown in

.

Preparation of the environment began with the creation of a fully updated Ubuntu 24.04 gold image. All system updates were applied to this base image to ensure consistency across all subsequent virtual machines. From this gold image, linked clones were created for the Ansible control node and managed servers. Linked cloning ensured minimal disk usage while maintaining identical baseline configurations across all systems. The NAT network used for this laboratory operated on the 192.168.176.0/24 subnet, with a default gateway at 192.168.176.2 and external DNS resolution provided by 9.9.9.9.

Static IP addressing was then configured on each Linux server to ensure predictable network communication, which is essential for configuration management tools such as Ansible. Network configuration was performed using Net plan, with the active configuration file located in `/etc/netplan` (Canonical Ltd. 2024). Prior to modification, a backup of the existing configuration file was created to preserve the original DHCP-based setup. The Net plan configuration was updated to disable DHCP, assign a fixed IPv4 address, define the default route via the NAT gateway, and specify DNS resolvers as seen in Figure 2. To prevent cloud-init from overwriting the static network configuration on reboot, cloud-init network configuration was explicitly disabled by creating a configuration file within `/etc/cloud/cloud.cfg.d`, see Figure 3. This step was necessary due to Ubuntu's default cloud-init behaviour in modern releases and ensured persistence of the static network settings.

Following successful network configuration, system hostnames were configured by editing the `/etc/hostname` file on each node. As no internal DNS server was available within the test environment, hostname resolution was implemented manually using the `/etc/hosts` file as seen in Figure 4. This allowed each system to resolve the hostnames of other nodes within the environment, providing a simple but reliable alternative to DNS for small-scale or laboratory-based infrastructures. Connectivity between nodes was verified using standard network testing tools to ensure that both IP-level communication and hostname resolution were functioning correctly shown in Figure 5.

Secure communication between the Ansible control node and the managed servers was established using SSH key-based authentication. An SSH server was installed on the control node where required, and a local RSA key pair was generated for use with Ansible. The public key was copied to each managed server using `ssh-copy-id`, enabling password less authentication as shown in Figure 6. While all tasks in this laboratory were carried out using a personal user account for simplicity, this is acknowledged as poor practice in production environments (OpenBSD Project 2023). In real-world deployments, a dedicated service account with restricted privileges and controlled sudo access would be used for automation tasks to improve security and auditability.

Once secure connectivity was confirmed, Ansible was installed on the control node using the system package manager. Following installation, the Ansible inventory was configured using a YAML-based inventory file, grouping managed hosts according to operating system version. Due to the use of Ubuntu 24.04, which defaults to Python 3.12, the Python interpreter was explicitly defined within the inventory configuration to ensure compatibility with Ansible modules. (Red Hat 2024) shown in Figure 7. This step was required to avoid interpreter mismatches and reflects best practice when managing heterogeneous or modern Linux environments. The inventory configuration was validated using Ansible's built-in inventory listing functionality to confirm correct parsing and host group membership.

Initial testing of the Ansible environment was performed using the Ansible ping module, which verified end-to-end connectivity, authentication, and interpreter configuration between the control node and managed servers. Following successful verification, arbitrary shell commands were executed remotely across the managed hosts to demonstrate Ansible's ability to perform parallel command execution. Disk usage analysis was conducted using standard Linux utilities, illustrating how Ansible can be used for system auditing and information gathering at scale.

The final phase of the laboratory involved the creation and execution of Ansible playbooks. Playbooks were written in YAML and stored within the user's home directory on the control node

see Figure 8, Figure 9, Figure 10. Privilege escalation was required for system-level tasks, and sudo credentials were provided at runtime using Ansible's become functionality. A system update playbook was implemented to perform distribution upgrades, refresh package caches, detect whether reboots were required, and remove unused dependencies. The playbook demonstrated Ansible's idempotent behaviour, applying changes only when necessary and leaving systems unchanged if they were already compliant. Additional playbooks were created to install and manage services such as Apache and MariaDB, further reinforcing the role of Ansible in repeatable service deployment and configuration management (Humble and Farley 2010).

Results and Testing

As agreed with the lecturer, due to configuration challenges with azure labs, results and testing could not be completed.

Conclusions

This laboratory explored the configuration and use of Ansible as an automation and configuration management tool within a controlled, virtualised environment. The laboratory demonstrated the core principles underpinning modern Infrastructure as Code practices by preparing Linux systems, implementing static networking, establishing secure SSH authentication, and using structured inventories and playbooks. The work highlighted how Ansible enables the centralised management of multiple systems using declarative configuration files, supporting repeatability, consistency, and reduced administrative overhead.

Despite designing and implementing the full Ansible workflow, the Azure laboratory environment did not allow for comprehensive end-to-end testing. The Azure tools available for this module did not support the deployment of multiple interconnected virtual machines in a manner required for Ansible's control-node and managed-host architecture. As a result, certain verification steps, including live execution of playbooks across multiple target nodes, were limited. This constraint was environmental rather than technical and did not reflect a lack of understanding of Ansible or its operational requirements (Microsoft 2024).

Despite these limitations, the laboratory achieved its intended learning outcomes by demonstrating a clear understanding of Ansible's architecture, configuration processes, and operational logic. The structured approach taken to inventory management, interpreter configuration, and playbook design reflects best-practice guidance for automation tools and aligns with industry-recognised Infrastructure as Code methodologies. The design decisions made throughout the laboratory illustrate how Ansible would operate effectively within a fully supported virtual or on-premises environment.

In conclusion, this laboratory gave helpful information about configuration management and automation at scale, reinforcing the role of Ansible as a key tool within modern DevOps workflows. While practical testing was constrained by the limitations of the Azure environment, the theoretical implementation and documented configurations demonstrate readiness for deployment in more suitable infrastructures such as VMware, Hyper-V, or cloud platforms with full virtual machine

support. This laboratory has developed skills that provide a solid foundation for further exploration of automation, orchestration, and scalable system administration (Microsoft 2025).

References

- Canonical Ltd. (2024) Netplan Configuration Reference [online], available:
<https://netplan.io/reference>.
- Humble, J. and Farley, D. (2010) *Continuous Delivery: Reliable Software Releases through Build, Test, and Deployment Automation*, Boston: Addison-Wesley.
- Microsoft (2024) Azure Virtual Machines Documentation [online], *Microsoft Learn*, available:
<https://learn.microsoft.com/en-us/azure/virtual-machines/>.
- Microsoft (2025) DevOps Resource Center [online], *Microsoft Learn*, available:
<https://learn.microsoft.com/en-us/devops/>.
- OpenBSD Project (2023) OpenSSH Manual Pages [online], available:
<https://www.openssh.com/manual.html>.
- Red Hat (2024) Getting Started with Ansible [online], *Ansible Documentation*, available:
https://docs.ansible.com/ansible/latest/getting_started/index.html.

Appendices

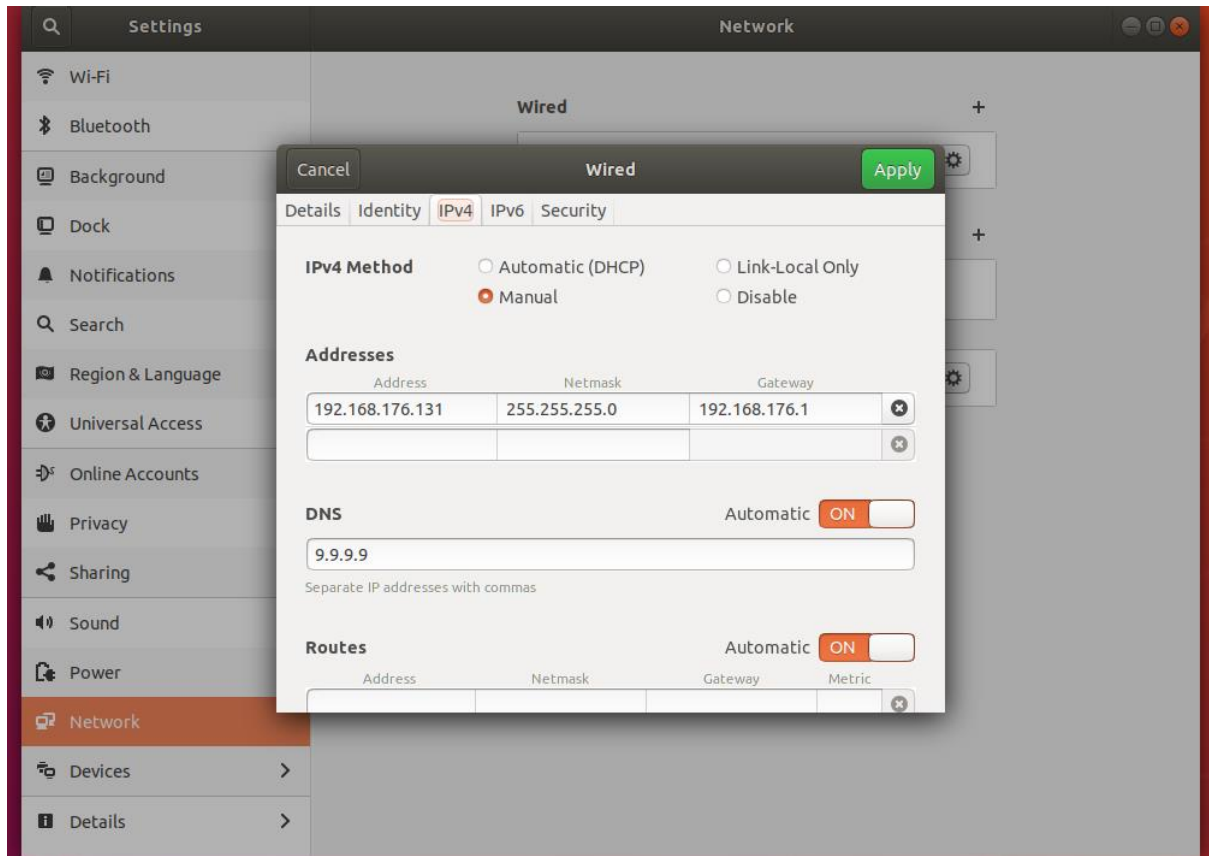
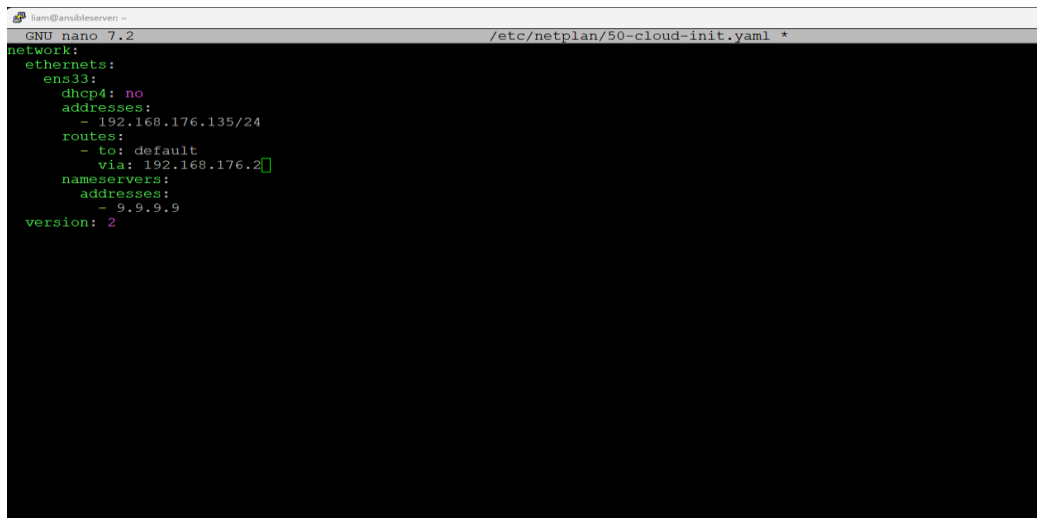


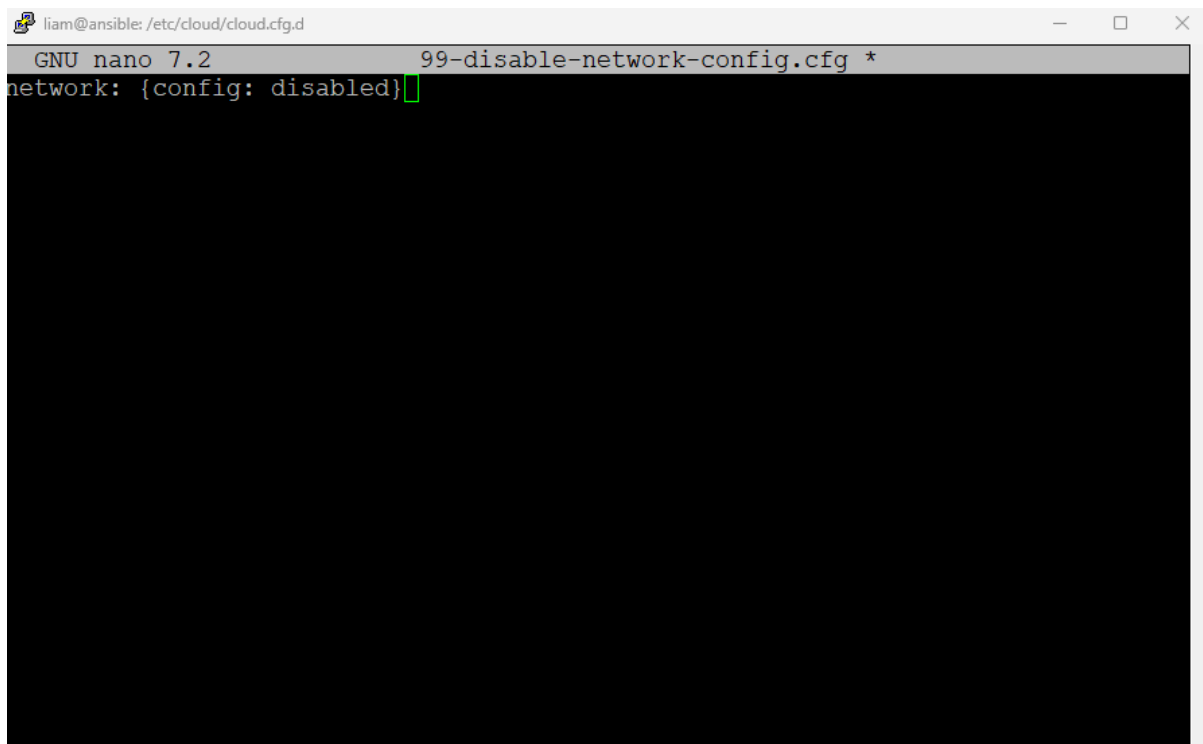
Figure 1:Ansible Manager IPv4 address

Laboratory Report



```
liam@ansibleserver ~  
GNU nano 7.2 /etc/netplan/50-cloud-init.yaml *  
network:  
  ethernet:  
    ens33:  
      dhcp4: no  
      addresses:  
        - 192.168.176.135/24  
      routes:  
        - to: default  
          via: 192.168.176.2  
      nameservers:  
        addresses:  
          - 9.9.9.9  
version: 2
```

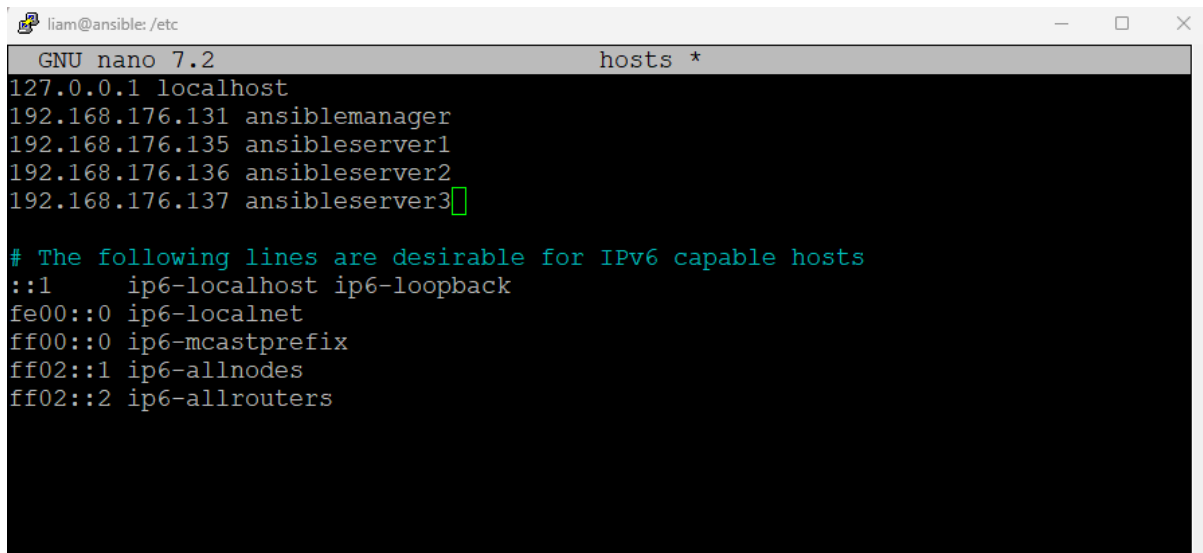
Figure 2: Net plan static IP configuration



```
liam@ansible: /etc/cloud/cloud.cfg.d  
GNU nano 7.2 99-disable-network-config.cfg *  
network: {config: disabled}
```

Figure 3: Disable Network Config file

Laboratory Report

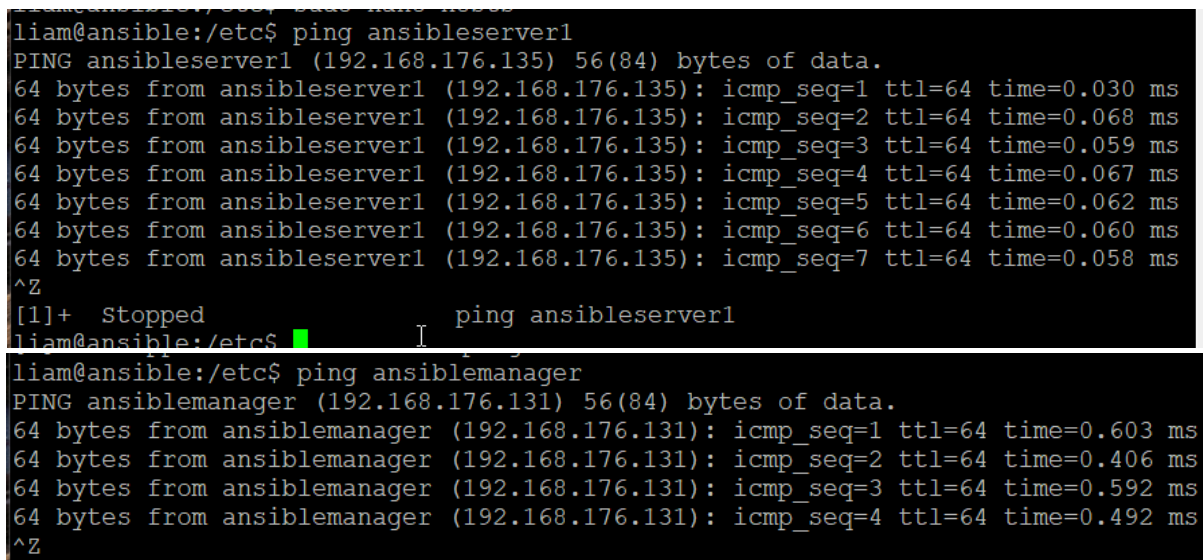


The screenshot shows a terminal window with the title bar 'liam@ansible: /etc'. The window contains the GNU nano 7.2 editor editing the /etc/hosts file. The file content is as follows:

```
127.0.0.1 localhost
192.168.176.131 ansiblemanager
192.168.176.135 ansibleserver1
192.168.176.136 ansibleserver2
192.168.176.137 ansibleserver3

# The following lines are desirable for IPv6 capable hosts
::1          ip6-localhost ip6-loopback
fe00::0 ip6-localnet
ff00::0 ip6-mcastprefix
ff02::1 ip6-allnodes
ff02::2 ip6-allrouters
```

Figure 4: /etc/hosts hostname resolution



The screenshot shows a terminal window with the title bar 'liam@ansible: /etc\$'. The user has performed two ping tests. The first test is to 'ansibleserver1' (192.168.176.135), showing 7 successful pings with times ranging from 0.030 ms to 0.068 ms. The second test is to 'ansiblemanager' (192.168.176.131), showing 4 successful pings with times ranging from 0.406 ms to 0.603 ms.

```
liam@ansible:/etc$ ping ansibleserver1
PING ansibleserver1 (192.168.176.135) 56(84) bytes of data.
64 bytes from ansibleserver1 (192.168.176.135): icmp_seq=1 ttl=64 time=0.030 ms
64 bytes from ansibleserver1 (192.168.176.135): icmp_seq=2 ttl=64 time=0.068 ms
64 bytes from ansibleserver1 (192.168.176.135): icmp_seq=3 ttl=64 time=0.059 ms
64 bytes from ansibleserver1 (192.168.176.135): icmp_seq=4 ttl=64 time=0.067 ms
64 bytes from ansibleserver1 (192.168.176.135): icmp_seq=5 ttl=64 time=0.062 ms
64 bytes from ansibleserver1 (192.168.176.135): icmp_seq=6 ttl=64 time=0.060 ms
64 bytes from ansibleserver1 (192.168.176.135): icmp_seq=7 ttl=64 time=0.058 ms
^Z
[1]+  Stopped                  ping ansibleserver1
liam@ansible:/etc$

liam@ansible:/etc$ ping ansiblemanager
PING ansiblemanager (192.168.176.131) 56(84) bytes of data.
64 bytes from ansiblemanager (192.168.176.131): icmp_seq=1 ttl=64 time=0.603 ms
64 bytes from ansiblemanager (192.168.176.131): icmp_seq=2 ttl=64 time=0.406 ms
64 bytes from ansiblemanager (192.168.176.131): icmp_seq=3 ttl=64 time=0.592 ms
64 bytes from ansiblemanager (192.168.176.131): icmp_seq=4 ttl=64 time=0.492 ms
^Z
```

Figure 5: Network connectivity test (ping)

```
liam@AnsibleManager:/etc$ ssh-keygen -t rsa -b 4096 -f ~/.ssh/local.rsa -C l00187849@atu.ie
Generating public/private rsa key pair.
Enter passphrase (empty for no passphrase):
Enter same passphrase again:
Your identification has been saved in /home/liam/.ssh/local.rsa.
Your public key has been saved in /home/liam/.ssh/local.rsa.pub.
The key fingerprint is:
SHA256:8bBAK+EanRQJUnZkSdHg1hZ2tDyqB2cfPLV1ifJQpmM l00187849@atu.ie
The key's randomart image is:
+---[RSA 4096]-----+
| ..++&Xo.o .o. + |
| o 0+=.+ +o. o. |
| . B = +E+ . |
| + o *...o |
| . . S o . |
| = o o |
| . . + |
| . . |
+-----[SHA256]-----+
liam@AnsibleManager:/etc$
```

Figure 6: SSH key generation

```
liam@AnsibleManager:/etc/ansible$ ansible-inventory --list -y
all:
  children:
    ub2404:
      hosts:
        AnsibleManager: {}
        ansibleserver: {}
    ungrouped:
      hosts:
        editing: {}
liam@AnsibleManager:/etc/ansible$
```

Figure 7: Ansible YAML inventory file

```
liam@AnsibleManager:~$ more update_ub2404.yml
---
- name: Updates
  hosts: all
  gather_facts: yes
  become: yes

  tasks:
    - name: Perform a dist-upgrade.
      ansible.builtin.apt:
        upgrade: dist
        update_cache: yes

    - name: Check if a reboot is required.
      ansible.builtin.stat:
        path: /var/run/reboot-required
        get_checksum: no
        register: reboot_required_file

    - name: Reboot the server (if required).
      ansible.builtin.reboot:
        when: reboot_required_file.stat.exists == true

    - name: Remove dependencies that are no longer required.
      ansible.builtin.apt:
        autoremove: yes
liam@AnsibleManager:~$
```

Figure 8: Update Playbook code

```
liam@AnsibleManager:~$ more InstallApache.yml
- name: Install Apache
  hosts: webserver
  become: true

  tasks:
    - name: Install Apache
      apt:
        name: apache2
        state: present

    - name: Start Apache
      service:
        name: apache2
        state: started
        enabled: yes
liam@AnsibleManager:~$
```

Figure 9: Install Apache Web Server Code

```
liam@AnsibleManager:~$ more InstallMariaDB.yml
- name: Install MariaDB
  hosts: mariadbserver
  become: true

  tasks:
    - name: Install MariaDB
      apt:
        name: mariadb-server
        state: present

    - name: Start MariaDB
      service:
        name: mariadb
        state: started
        enabled: yes
liam@AnsibleManager:~$
```

Figure 10: install MariaDB Code